Network Emulation with tc

Custom OS images · Traffic Control · Hands-on Containerlab exercises

1

Custom OS Images

Why alpine:latest isn't enough — and how to fix it

Why We Need a Custom Image

alpine:latest gives you a tiny OS — but it is missing almost every networking tool you need.

❌ bare alpine:latest

`tcpdump`	Packet capture — not installed
`iperf3`	Bandwidth measurement — missing
`traceroute`	Path tracing — missing
`tc`	Traffic control — missing (iproute2)
`bash`	Only /bin/sh, no bash
`dig / nslookup`	DNS tools — missing

✅ alpine-nettools (our custom image)

`bash`	Full shell — scripts, tab completion
`iproute2`	ip, ss, tc — the full toolkit
`tcpdump`	Live packet capture
`iperf3`	TCP/UDP throughput testing
`traceroute`	Hop-by-hop path tracing
`bind-tools`	dig, nslookup for DNS queries
`curl / wget`	HTTP testing
`net-tools`	ifconfig, netstat (legacy compat)

Building alpine-nettools — The Dockerfile

```
# Dockerfile
FROM alpine:latest

RUN apk update && apk add --no-cache \
    bash          \   # full shell
    iproute2      \   # ip, ss, tc
    iputils       \   # ping (modern)
    tcpdump       \   # packet capture
    traceroute    \   # path tracing
    iperf3        \   # bandwidth tests
    net-tools     \   # ifconfig, netstat
    bind-tools    \   # dig, nslookup
    curl wget        # HTTP tools
```

FROM alpine:latest

Start from the official Alpine base — tiny, secure, and fast to pull. Our additions cost only ~35 MB extra.

apk add --no-cache

Alpine's package manager. --no-cache skips the local index cache, keeping the image size minimal.

iproute2 is the key package

This single package gives you 'ip', 'ss', AND 'tc'. Without it you cannot do any traffic shaping.

Build and test:

```
docker build -t alpine-nettools:latest .
docker run --rm -it alpine-nettools:latest sh
/ # tc -V          # tc utility, iproute2-6.17.0
/ # iperf3 --version   # iperf 3.19.1
```

Referencing Your Custom Image in Containerlab YAML

Before — bare alpine

```
topology:
  nodes:
    client:
      kind: linux
      image: alpine:latest    # ← no tc, no
iperf3
      exec:
        - ip addr add 10.0.1.1/24 dev eth1
```

After — nettools image

```
topology:
  nodes:
    client:
      kind: linux
      image: alpine-nettools:latest  # ← full
toolkit
      exec:
        - ip addr add 10.0.1.1/24 dev eth1
```

Three ways to supply the image:

Local build (workshop)

```
docker build -t alpine-
nettools:latest .
```

Build once on the host. Containerlab picks it up automatically — no registry needed.

Pre-built registry image

```
image:
ghcr.io/myorg/nettools:latest
```

Push to Docker Hub or GHCR. Useful when every trainee needs the same image without building locally.

Inline exec install

```
exec:
  - apk add --no-cache tcpdump
```

Install packages at startup via exec. Slower (downloads every deploy) but requires no custom image.

2

Traffic Control – tc

Simulating real-world network conditions in software

What is tc? — The Linux Traffic Control Subsystem

tc is the command-line interface to the Linux kernel's built-in traffic control (QoS) engine.
It can artificially add latency, loss, corruption, reordering, and bandwidth limits to any network interface.

💡 Why do we need it? A Containerlab virtual link is essentially a loopback — zero latency, zero loss, infinite bandwidth. Real networks are nothing like that. tc lets us inject realistic impairments so our experiments actually test protocol behaviour.

Latency

Add a fixed delay (e.g. 50 ms) or a variable jitter (±10 ms) to emulate WAN links, satellite, or congested paths.

Packet Loss

Drop a percentage of packets randomly (e.g. 5%) or in bursts to simulate noisy wireless or failing hardware.

Bandwidth

Cap the throughput on a link to simulate DSL, 4G, or a metered WAN circuit. Uses TBF queueing disciplines.

Reorder / Corrupt

Deliver packets out-of-order or flip random bits — useful for testing TCP recovery and checksum handling.

tc netem — Network Emulator Qdisc

netem runs in the kernel — zero overhead. One command is all you need.

```
# Fixed one-way delay
tc qdisc add dev eth1 root netem delay 50ms

# Delay  ±  jitter  (uniform spread)
tc qdisc add dev eth1 root netem delay 50ms 10ms

# Delay  ±  jitter  (normal distribution — more realistic)
tc qdisc add dev eth1 root netem delay 50ms 10ms distribution
normal

# Packet loss  (5% of packets dropped randomly)
tc qdisc add dev eth1 root netem loss 5%

# Combine delay + loss in one command
tc qdisc add dev eth1 root netem delay 50ms loss 2%

# Modify an existing rule in-place
tc qdisc change dev eth1 root netem delay 100ms

# Remove ALL tc rules from eth1
tc qdisc del dev eth1 root
```

⏱ Delay is one-way

50 ms on eth1 adds 50 ms in that direction only.
For symmetric RTT add the same rule on the return interface.

🎲 add vs change vs del

'add' first time · 'change' to update without resetting flows · 'del' to remove everything

Combine freely

delay, loss, corrupt, duplicate and reorder can all be combined in one command.

Anatomy of a tc netem Command

```
tc qdisc add dev eth1 root netem delay 50ms 10ms
```

command — *tc — traffic control CLI*

action — *add/change a new qdisc*

interface — *apply to eth1*

attachment — *root = top-level*

qdisc type — *netem = network emulator*

base delay — *50 ms one-way*

± jitter — *±10 ms spread*

All netem parameters:

`delay <time>`
Add fixed one-way delay e.g. delay 50ms

`delay <time> <jitter>`
Add delay ± jitter e.g. delay 50ms 10ms

`delay … distribution normal`
Normal (Gaussian) spread rather than uniform

`loss <percent>`
Drop packets randomly e.g. loss 5%

`corrupt <percent>`
Flip random bits in the packet payload

`duplicate <percent>`
Clone and deliver extra copies of packets

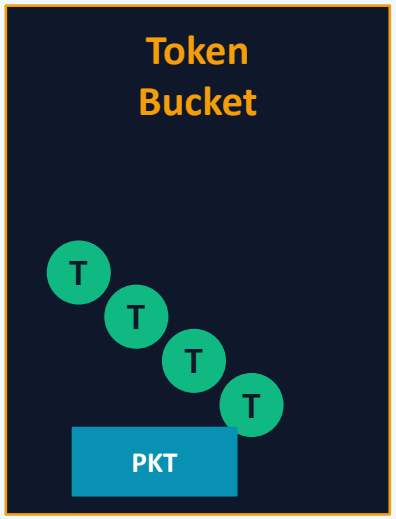
`reorder <percent>`
Deliver a fraction of packets out of sequence

`rate <speed>`
Soft rate limit inside netem queue e.g. rate 10mbit

`limit <packets>`
Maximum queue depth before tail-drop

TBF — Token Bucket Filter: Bandwidth Limiting

TBF caps the total throughput of an interface to a fixed, sustained rate.



← tokens added at 'rate'

'burst' = max tokens at once

needs 1 token or it waits

```
# Limit eth1 to 10 Mbit/s
tc qdisc add dev eth1 root tbf
    rate 10mbit       # sustained throughput cap
    burst 32kbit      # token bucket size (allow short bursts)
    latency 50ms      # drop packets queued longer than this

# 1 Mbit/s — DSL / slow link simulation
tc qdisc add dev eth1 root tbf rate 1mbit \
burst 32kbit \
latency 100ms

# Remove
tc qdisc del dev eth1 root
```

rate — Throughput cap: 1mbit · 500kbit · 10mbit · 1gbit

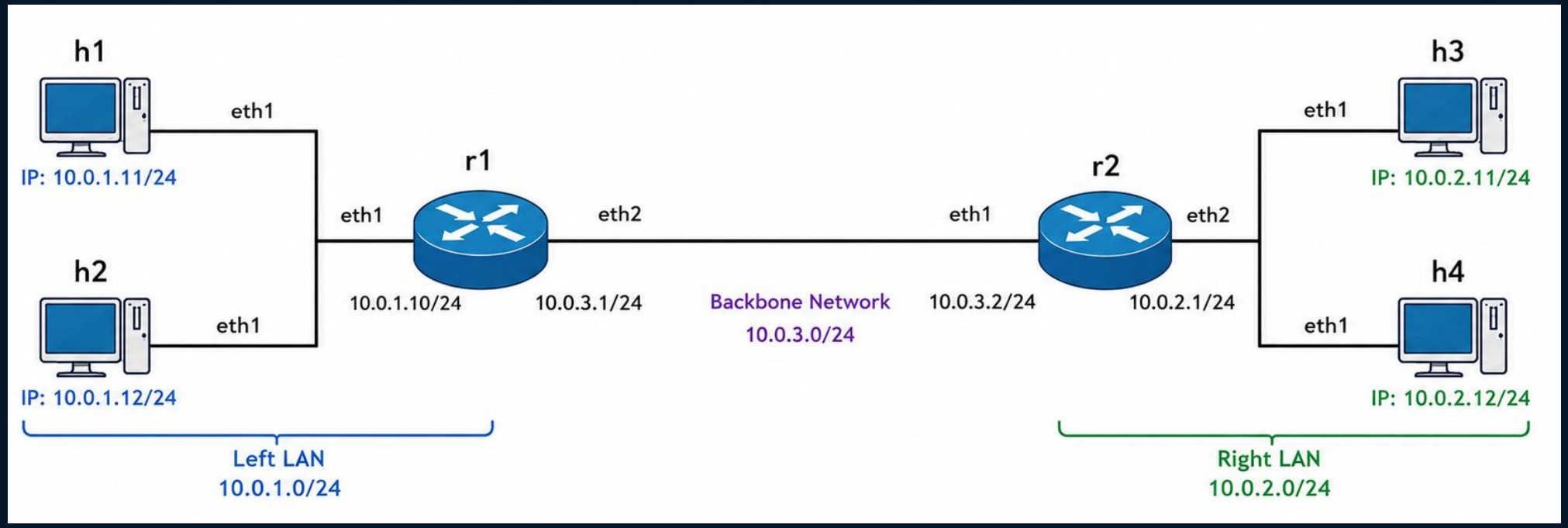
burst — Max bytes at wire speed before cap kicks in — 32kbit works for most labs

latency — Packets waiting longer than this are dropped (acts as queue depth limit)

tc Quick-Reference Cheatsheet

`tc qdisc show dev ETH`	List active qdiscs on interface ETH
`tc qdisc del dev ETH root`	Remove ALL tc rules from interface ETH
`tc qdisc add dev ETH root netem delay Xms`	Add X ms one-way latency
`tc qdisc add dev ETH root netem delay Xms Yms`	Add X ms delay ± Y ms jitter
`tc qdisc add dev ETH root netem loss X%`	Drop X% of packets randomly
`tc qdisc add dev ETH root netem delay Xms loss Y%`	Combine: latency + loss in one rule
`tc qdisc change dev ETH root netem delay Xms`	Modify existing netem rule in-place
`tc qdisc add dev ETH root tbf rate Xmbit \ burst 32kbit latency 50ms`	Limit throughput to X Mbit/s (TBF)
`ping -c 20 IP`	Verify latency — compare avg RTT before/after
`iperf3 -s  /  iperf3 -c IP -t 10`	Verify bandwidth — server then client
`iperf3 -c IP -u -b 5M -t 10`	Verify loss — UDP mode reports loss %

Assignment – tc on the Dumbbell Topology



1 Deploy with custom image

Rewrite your dumbbell YAML so every node uses alpine-nettools:latest. Deploy and confirm all nodes are running with clab inspect.

2 Baseline measurements

From h1, ping h3 and record the avg RTT. Then run iperf3 from h1 to h3 (start iperf3 -s on h3 first). Record the baseline throughput in Mbit/s.

3 Emulate a WAN backbone

On r1's eth2 (backbone-facing), apply: 50ms delay + 1% loss + 10mbit rate cap. Command:
tc qdisc add dev eth2 root handle 1: tbf rate 10mbit burst 32kbit latency 100ms
tc qdisc add dev eth2 parent 1:1 handle 10: netem delay 50ms loss 1%

4 Re-measure and compare

Repeat the ping and iperf3 tests. Ping RTT should increase by ~50ms. iperf3 throughput should drop to ~10mbit. UDP iperf3 should show ~1% loss.

5 Verify with tcpdump

While h1 pings h3, run tcpdump on r2's eth1. You should see ICMP packets arriving at the rate-limited, delayed pace. Count the inter-packet gap.

6 Clean up and explain

Remove the tc rules with 'tc qdisc del dev eth2 root'. Re-run iperf3 and confirm throughput returns to baseline. Be prepared to explain what TBF and netem each contributed.

Summary — What We Covered

Build a custom alpine-nettools image with Dockerfile — one RUN line installs all tools including iproute2 (which provides tc).

TBF caps a single interface to a fixed rate. Simple, predictable, best for most exercises.

Reference it in any YAML with 'image: alpine-nettools:latest'. No other change needed.

TBF + netem can be chained: use a handle on TBF and attach netem as a child for combined rate limiting and latency.

tc netem adds latency, jitter, and loss. One command, zero cost, instant effect.

Always verify: 'tc qdisc show' to read rules, ping for latency, iperf3 for throughput and loss.

Next: apply these techniques to your dumbbell assignment and explore more netem parameters.